ENGR Module 8-9 | Quiz review

24 MCQs + 8 short FRQs + 3 coding challenges

Based on Lectures 8 and 9, with the earlier graded quizzes and Topics 1-7 review as the difficulty baseline. New questions combine the new concepts with previously learned loops, conditions, types, and indexing.

Start with the two short review pages, then attempt the questions before opening the separate answer key. Every MCQ has exactly one correct choice. Assume Python 3 and fresh state for each snippet. Where an output choice uses / to separate lines, / is notation for a newline, not a printed character.

Coverage

Module 8: top-down design, hierarchy vocabulary and tradeoffs, dictionary creation, updates, key membership, and direct iteration. Module 9: function calls and definitions, docstrings, tuple returns, scope, defaults, return flow, globals, and mutation versus rebinding.

Tools and boundaries

Use the syntax from the two lectures plus Topics 1-7. No classes, recursion, comprehensions, exceptions as control flow, keyword-only parameters, or mutable-default tricks. Dictionary methods such as get/items/values are not needed. Functions in the coding section must return results rather than print them.

Suggested practice plan

15 minutes: skim both module guides.

45-60 minutes: MCQs with handwritten state tables.

35-45 minutes: short FRQs.

75-105 minutes: three coding challenges.

30 minutes: grade, retrace mistakes, and test boundary cases.

Source map

Lecture8.pdf: pp11-18 design; pp19-25 study analyzer; pp28-32 dictionaries. Lecture9.pdf: pp9-30 function interfaces, tuples, docstrings; pp39-51 execution; pp63-88 scope/defaults/return; pp93-107 shared objects. Earlier Module 2/3/4/6 submissions and Topics 1-7 review inform question style and difficulty. Page numbers refer to PDF pages.

All questions are newly written practice, not predictions of your instructor's exact quiz. The answer key contains explanations, short-answer scoring guidance, and two hints plus a reference solution for each coding challenge.

Module 8 | Top-down design and dictionaries

Top-down design

Start with the whole goal, divide it into coherent major tasks, and refine each task until its implementation is clear. Sibling tasks should have distinct responsibilities. Several reasonable decompositions may exist.

Tree vocabulary

Root: no parent. Node: any element. Parent/child: adjacent levels. Leaf: no children. A leaf does not have to be on the deepest level. Hierarchies manage complexity but can hide connections across branches.

Dictionary basics

A dictionary maps keys to values. Keys are unique; values may repeat. d[key] looks up a key, not a position. Assigning an existing key replaces its value; assigning a new key adds a pair. Accessing a missing key raises KeyError.

Membership and loops

key in d checks exact keys, not values or substrings. for key in d iterates over keys; use d[key] to read each corresponding value. len(d) counts pairs. Update existing values freely; avoid adding/deleting keys while directly iterating over the dictionary.

Counting template

Initialize a missing key once; update on every occurrence. Do not reset a count each time you see the key.

Scope of this review

Lecture 8 teaches {}, key indexing/assignment, membership, and direct iteration. This bank does not require get, items, values, pop, dictionary comprehensions, or other methods from a broader dictionary guide.

```
counts = {}
for word in words:
    if word not in counts:
        counts[word] = 0
    counts[word] += 1
```

Source: Lecture8.pdf pp11-18,19-25,28-32

Module 9 | Functions, scope, and shared objects

Define, call, return

def records a function; its body runs when called. Parameters are the local names in the header; arguments are supplied at the call. return immediately exits the function. print displays text; it does not substitute for returning a result. Reaching the end without return gives None.

Tuples and multiple results

return a, b returns one tuple. x, y = f() unpacks by position. Tuple indexing and slicing work, but replacing a tuple slot raises TypeError. A mutable list stored inside a tuple can still change.

Scope

Each call gets local parameter/variable bindings. Reassigning a numeric or string parameter does not change the caller variable. A function can read a global if not shadowed; assigning to a name makes it local unless declared global. Reading such a local before assigning it raises UnboundLocalError.

Mutation versus rebinding

a[0] = 9 changes a shared list; a = [9] only rebinds local a. For lists, a += [9] mutates while a = a + [9] rebinds. Dictionaries also share mutations through passed references. b = a aliases a list; b = a[:] makes a new outer list, not a recursive copy of nested lists.

Defaults and documentation

Omitted trailing arguments use defaults; an explicit zero does not. In the taught positional headers, required parameters precede defaulted ones. Put a docstring first in the body describing purpose, inputs, outputs, and relevant units. Prefer arguments and returned results over hidden global dependencies.

Trace checklist

Bind arguments -> run the body in order -> track shared-object mutations separately -> stop at return -> assign/use the returned object in the caller. Keep a separate row of local variables for every call.

```
def split_total(total, people=2):
    """Return equal integer share and leftover units."""
    return total // people, total % people
share, leftover = split_total(17, 5)  # 3, 2
```

Source: Lecture9.pdf pp9-30,39-51,63-88,93-107

Multiple choice | Q1-Q2

Q1 | Top-down design

Which decomposition best follows top-down design for a study-session analyzer?

A. Create separate tasks for each line of Python syntax.

B. Choose every variable name before identifying the major tasks.

C. Read sessions; answer queries; report results. Then refine each task.

D. Write one large loop, then divide it into equal-length pieces.

Answer: ___ Reason / trace:

Q2 | Hierarchies

A design tree has root R with children A and B. A has children C and D; B has no children. Which statement is correct?

A. B, C, and D are leaves; A is the parent of C and D.

B. Only C and D are leaves.

C. Every node must have exactly two children.

D. R has no children because it is the root.

Answer: ___ Reason / trace:

Multiple choice | Q3-Q4

Q3 | Design judgment

Which claim about top-down design agrees with the lecture?

A. Several decompositions can be reasonable; hierarchies may miss cross-connections.

B. Every problem has exactly one correct design tree.

C. Stop only when every node corresponds to exactly one line of code.

D. A hierarchy captures every relationship in a system.

Answer: ___ Reason / trace:

Q4 | Dictionary updates

What is printed?

```
d = {"A": 2, "B": 5}
d["A"] = d["B"] - 1
d["C"] = d["A"] + d["B"]
d["B"] = 0
print(len(d), d["A"], d["C"])
```

A. 3 0 4

B. 4 4 9

C. 3 4 4

D. 3 4 9

Answer: ___ Reason / trace:

Multiple choice | Q5-Q6

Q5 | Dictionary membership

Which expression is True?

```
d = {"ab": 3, "cab": 3}
```

A. 3 in d

B. "ab" in d and 3 not in d

C. "ab" not in d

D. "a" in d

Answer: ___ Reason / trace:

Q6 | Key versus position

What happens?

```
d = {"0": 7, 1: 9}
print(d["0"], d[1])
print(d[0])
```

A. Prints 7 9, then 9

B. Raises IndexError before any output

C. Prints 7 9, then 7

D. Prints 7 9, then raises KeyError

Answer: ___ Reason / trace:

Multiple choice | Q7-Q8

Q7 | Dictionary accumulation

What is printed?

```
totals = {}
for name in ["A", "B", "A", "A"]:
    if name not in totals:
        totals[name] = 0
    totals[name] += 2
print(totals["A"], totals["B"], len(totals))
```

A. 6 2 4

B. 8 2 2

C. 2 2 2

D. 6 2 2

Answer: ___ Reason / trace:

Q8 | Dictionary traversal

What is printed?

```
d = {"a": 2, "b": 4}
total = 0
for key in d:
    d[key] += 1
    total += d[key]
print(total, d["a"], d["b"])
```

A. 8 3 5

B. 6 3 5

C. TypeError

D. 6 2 4

Answer: ___ Reason / trace:

Multiple choice | Q9-Q10

Q9 | Definition versus execution

What is printed?

```
def unused():
    missing_function()
def plus_one(x):
    return x + 1
print(plus_one(4))
```

A. 4

B. NameError before output

C. 5

D. None

Answer: ___ Reason / trace:

Q10 | Print versus return

What is printed? The symbol / in the choices means a new line.

```
def double(x):
    print(x * 2)
y = double(3)
print(y)
```

A. 6 only

B. 6 / None

C. None / 6

D. 6 / 6

Answer: ___ Reason / trace:

Multiple choice | Q11-Q12

Q11 | Immediate return

What is printed?

```
def first_even(values):
    for value in values:
        if value % 2 == 0:
            return value
    return -1
print(first_even([3, 8, 2]), first_even([1, 5]))
```

A. 8 None

B. 8 -1

C. 2 -1

D. -1 -1

Answer: ___ Reason / trace:

Q12 | Tuple return and unpacking

What is printed?

```
def parts(a, b):
    return a // b, a % b
r, q = parts(17, 5)
print(r, q, r + q)
```

A. 2 3 5

B. TypeError

C. 3.4 2 5.4

D. 3 2 5

Answer: ___ Reason / trace:

Multiple choice | Q13-Q14

Q13 | Defaults

What is printed?

```
def calc(a, b=2, c=1):
    return a * b + c
print(calc(3), calc(3, 4), calc(3, 4, 0))
```

A. 7 13 12

B. 7 10 12

C. 6 12 0

D. 7 12 12

Answer: ___ Reason / trace:

Q14 | Function headers

Which header is invalid for the ordinary positional parameters taught in the slides?

A. def f(a=1, b):

B. def f(a, b):

C. def f(a=1, b=2):

D. def f(a, b=1):

Answer: ___ Reason / trace:

Multiple choice | Q15-Q16

Q15 | Local state

What is printed?

```
def bump(a):
    a += 3
    return a
a = 5
b = bump(a)
print(a, b, bump(b))
```

A. 11 8 11

B. 5 5 8

C. 8 8 11

D. 5 8 11

Answer: ___ Reason / trace:

Q16 | Reading global state

What is printed?

```
rate = 2
def cost(n):
    return n * rate
rate = 3
print(cost(4))
```

A. 8

B. NameError

C. UnboundLocalError

D. 12

Answer: ___ Reason / trace:

Multiple choice | Q17-Q18

Q17 | Assignment and scope

What happens?

```
score = 10
def add():
    score += 2
    return score
print(add())
```

A. 12

B. 10

C. UnboundLocalError before any output

D. None

Answer: ___ Reason / trace:

Q18 | Explicit global

What is printed?

```
count = 1
def step():
    global count
    count += 2
    return count
a = step()
b = step()
print(a, b, count)
```

A. 3 5 5

B. 5 5 5

C. 3 5 1

D. 3 3 1

Answer: ___ Reason / trace:

Multiple choice | Q19-Q20

Q19 | Mutation then rebinding

What is printed?

```
def change(a):
    a[0] = 9
    a = [4, 5]
    a[1] = 8
    return a
x = [1, 2]
y = change(x)
print(x, y)
```

A. [9, 2] [4, 8]

B. [9, 8] [4, 8]

C. [4, 8] [4, 8]

D. [1, 2] [4, 8]

Answer: ___ Reason / trace:

Q20 | List += versus +

What is printed?

```
def extend(a, b):
    a += [7]
    b = b + [8]
x = [1]
y = [2]
extend(x, y)
print(x, y)
```

A. [1] [2]

B. [1] [2, 8]

C. [1, 7] [2, 8]

D. [1, 7] [2]

Answer: ___ Reason / trace:

Multiple choice | Q21-Q22

Q21 | Copies and aliases

What is printed?

```
a = [1, 2]
b = a
c = a[:]
b[0] = 6
c[1] = 9
print(a, b, c)
```

A. [6, 2] [6, 2] [1, 9]

B. [1, 2] [6, 2] [1, 9]

C. [6, 2] [6, 2] [6, 9]

D. [6, 9] [6, 9] [6, 9]

Answer: ___ Reason / trace:

Q22 | Tuple containing a list

What happens? / means a new line.

```
a = [1, 2]
t = ("go", a)
a[0] = 9
print(t[1])
t[0] = "stop"
```

A. Prints [9, 2], then raises TypeError

B. Prints [9, 2], then changes t successfully

C. Raises TypeError before any output

D. Prints [1, 2], then raises TypeError

Answer: ___ Reason / trace:

Multiple choice | Q23-Q24

Q23 | Nested calls

What is printed?

```
def f(x):
    return x + 2
def g(x):
    return f(x) * x
print(f(g(3)))
```

A. 13

B. 25

C. 17

D. 15

Answer: ___ Reason / trace:

Q24 | Dictionary argument

What is printed?

```
def record(d, key):
    if key not in d:
        d[key] = 0
    d[key] += 1
    return d[key]
counts = {"x": 2}
a = record(counts, "x")
b = record(counts, "y")
print(a, b, counts["x"], len(counts))
```

A. 2 0 2 1

B. 3 1 3 2

C. 3 1 2 2

D. 3 1 2 1

Answer: ___ Reason / trace:

Short FRQ | F1-F2

F1 | Design before coding (6 points)

A program records study sessions (test name, minutes), answers repeated test-name queries, and prints a final summary. Draw or outline a top-down tree with three major tasks and at least two leaf tasks under each. Identify one parent and one leaf. Explain when to stop refining.

Response / code:

F2 | Repair dictionary counting (6 points)

The function should return counts for all words. Identify two logic defects and rewrite it. Example: ["red", "red", "blue"] must produce counts red=2 and blue=1.

```
def count_words(words):
    counts = {}
    for word in words:
        counts[word] = 0
        counts[word] += 1
        return counts
```

Response / code:

Short FRQ | F3-F4

F3 | Separate print from return (6 points)

State the output before failure and the error type. Rewrite area so result becomes 13 for this call, with no printing inside area. Add a one-sentence docstring.

```
def area(w, h):
    print(w * h)
result = area(3, 4) + 1
```

Response / code:

F4 | Parameter order and tuple results (6 points)

Write split_total(total, people=2) for nonnegative integer total and positive integer people. Return a tuple (equal_share, leftover) using integer division and remainder. Show one call using the default, unpack the result, and give the returned value for split_total(17, 5). Do not print inside the function.

Response / code:

Short FRQ | F5-F6

F5 | Repair scope without global (6 points)

Explain the failure. Rewrite the function and its call so score becomes 12 without global, input, or printing inside the function.

```
score = 10
def increase():
    score += 2
increase()
```

Response / code:

F6 | Trace two references (6 points)

Give the final values of x and y. Explain which statement affects the caller. Then change only the call so x stays [2, 4] while y has the same value as before.

```
def edit(a):
    a[0] += 3
    a = a + [9]
    return a
x = [2, 4]
y = edit(x)
```

Response / code:

Short FRQ | F7-F8

F7 | Early return or full scan? (6 points)

This function should return how many values meet the threshold. Explain why it fails, fix only the indentation of return, and give results for tally([8, 2, 9], 5) and tally([], 5).

```
def tally(values, threshold):
    count = 0
    for value in values:
        if value >= threshold:
            count += 1
        return count
```

Response / code:

F8 | Explain tuple immutability precisely (6 points)

Give the values printed. Explain why the list change is visible but the string reassignment is not. Would t[1] = [0] be allowed?

```
name = "A"
scores = [2, 4]
t = (name, scores)
name = "B"
scores[1] = 7
print(t[0], t[1])
```

Response / code:

Coding C1 | Study-session query engine

```
def analyze_sessions(names, minutes, queries):
```

Aggregate study sessions, then answer queries in their original order. Return a list of tuples (test_name, session_count, total_minutes). Repeated queries must produce repeated results. An unknown test returns a count and total of zero. Do not change any input list.

Constraints

names and minutes have equal lengths, 0-1000. Names and query strings are case-sensitive; minutes are nonnegative integers. queries has 0-1000 entries. Use a dictionary to aggregate each test once, then answer queries without rescanning all sessions. Use only dictionary indexing/membership, loops, list append, tuples, and ordinary arithmetic; no get/items/values, Counter, comprehensions, or imports.

Examples: call and returned result

```
analyze_sessions(["Math", "ENGR", "Math"], [20, 15, 30],
                 ["Math", "Chem", "Math"])
=> [("Math", 2, 50), ("Chem", 0, 0), ("Math", 2, 50)]
```

```
analyze_sessions([], [], ["ENGR"])
=> [("ENGR", 0, 0)]
```

```
analyze_sessions(["ENGR"], [0], ["ENGR"])
=> [("ENGR", 1, 0)]
```

Before coding

Write a short top-down plan and identify which objects may change. Include a docstring. Test empty input, repeated values/keys, and boundary cases. Hints and solutions are in the separate key.

Coding C2 | First unique reading

```
def first_unique(values):
```

Return the tuple (index, value) for the first element whose value occurs exactly once in the entire input list. First means smallest original index, not smallest numeric value. If no unique value exists, return (-1, 0). Do not modify values.

Constraints

0-1000 integer readings, including negatives and zero. Use a dictionary for frequencies and at most two full passes over values. Do not sort or use list count, dictionary get/items/values, sets, comprehensions, or imports.

Examples: call and returned result

```
first_unique([8, 8, 3, 5, 5])
=> (2, 3)
```

```
first_unique([4, 9, 4, 2])
=> (1, 9)
```

```
first_unique([0, 0])
=> (-1, 0)
```

```
first_unique([0])
=> (0, 0)
```

Before coding

Write a short top-down plan and identify which objects may change. Include a docstring. Test empty input, repeated values/keys, and boundary cases. Hints and solutions are in the separate key.

Coding C3 | Sequential stock requests

```
def process_requests(stock, names, amounts, apply=False):
```

Process requests in order. Accept a request only if its item is known and the current available stock is at least the requested amount. An accepted request reduces stock; a rejected one changes nothing. Return (accepted_flags, ending_stock). When apply is False, simulate on a new dictionary and leave stock unchanged. When apply is True, update the supplied stock dictionary itself and return that same dictionary as ending_stock.

Constraints

stock maps string names to nonnegative integers. names and amounts have equal lengths, 0-1000; amounts are strictly positive integers. Repeated items are allowed. Never create a stock entry for an unknown request. No printing, input, global, dictionary copy/get/items/values, imports, or comprehensions. Copy entries with a loop when simulating.

Examples: call and returned result

```
stock = {"bolt": 5, "nut": 2}
process_requests(stock, ["bolt", "bolt", "nut", "pin"], [3, 3, 2, 1])
=> ([True, False, True, False], {"bolt": 2, "nut": 0})
Original stock remains {"bolt": 5, "nut": 2}.
```

```
stock = {"bolt": 5}
process_requests(stock, ["bolt", "bolt"], [3, 2], True)
=> ([True, True], {"bolt": 0})
Original stock becomes {"bolt": 0}.
```

Before coding

Write a short top-down plan and identify which objects may change. Include a docstring. Test empty input, repeated values/keys, and boundary cases. Hints and solutions are in the separate key.